

Data Structures
Laboratory Sheet 2
Engineering - Instituto Superior de Engenharia de Coimbra

Binary Search and Variations

The method `criaArrayCom` (int value, int size, different boolean) shown here returns an ordered array of integers (all different if the corresponding parameter is true), with a specified dimension, which includes the indicated value.

```
static int[] criaArrayCom (
    int value,
    int dimension,
    boolean many different) {
    int m[] =new int[dimension];
    if(many different) {
        is(int i= 0;i<dimension;i++)
            m[i] =i* 10;
        if((value%10!= 0) || (0>value) || (value> (dimension-1) *
10))
            m[0] =value;
    }
    else{
        Random r=new Random ();
        int gamma= (Math.abs (value?) <10) 10: Math.abs (value);
        is(int i= 0;i<dimension;i++)
            m[i] =r.nextInt (gamma* 4) -gamma*2;
        m[0] =value;
    }
    Arrays.sort(m);
    return m;
}
```

Use this method to test the resolution of the following points.

- 1 - Build a method that performs a binary search recursively, returning true or false as the value is found or not.
- 2 - Build a method that performs a binary search iteratively, returning true or false as the value is found or not.
- 3 - Build a method that performs a binary search. This method should return the position where the value sought is found, or -1 if it is not in the array. Test the method.
- 4 - Build a method that performs a binary search. This method should return the position where the value sought is found, or a negative value (-X) if it is not found in the array. The value of $\text{abs}(X + 1)$ indicates the position where the searched value could be inserted to preserve the order.

Example:

array: {3,7,12,15}

searched result value = 15 = 3

Value = 3 searched result = 0

Value = 1 result sought = -1 (the value should be inserted at position 0)

Value = 4 searched result = -2 (the value should be inserted at position 1)
Value = 10 result sought = -3 (the value should be inserted at position 2)
searched result value = 13 = -4 (the value should be inserted at position 3)
Searched result value = 16 -5 = (the amount should be inserted at position 4)

To solve the following exercises, consider the binary search method and variants presented in the preceding paragraphs. All exercises should be solved with algorithms with complexity $O(\log N)$.

5 - Build a method that receives an ordered array of integers, all different, and a value, and returns the percentage of array values that are smaller than the indicated value.

Example:

array: {3,7,12,15}
Value = 15 = 0.75 result
Value = 14 = 0.75 result
Score = 3 Value = 0.0
Result = 1 Value = 0.0
Value = 100 result = 1.0

6 - Build a method that receives an ordered array of integers, all different, and two values defining a range. The method should indicate how many values of the array are within the specified range.

Example:

array: {3,7,12,15}
Values = (0.15) score = 4
Values = (3,7) result = 2
Values = (4.14) 2 = Result
Values = (4,5) result = 0
Values = (0.100) score = 4

7 - Build a method that takes an ordered array of integers, and a value. The method should indicate if this is repeated in the array.

Example:

array: 3,3,7,12,12,15 {}
Value = 15 result = false
Value = 14 result = false
Value = 12 result = true
Result Value = 3 = true

8 - Build a method that returns a parameter receiving an ordered array of integers, not repeated, and Z as a whole, and returns the largest element of the array smaller than Z (or Z if that element does not exist).

Example:

array: {3,7,12,15}
Result Value = 15 = 12
Result Value = 14 = 12
Result Value = 3 = 3
Result Value = 1 = 1
Result = 100 Value = 15

9 - Consider a method that takes an array of integers in which the numbers are arranged as follows: all negative numbers are found in higher indexes than positive numbers, and all the positive and negative numbers are ordered among themselves.

Example: {3,6,8, -10, -3, -2, -1}

Build a method that looks for a number in the array.

10 - Build a method that receives an array of integers ordered, non-repeated, and which returns the index of the first position in which the stored value is greater than the index (or -1 if this position does not exist).

Examples:

array: {3,7,12,15} result = 0

array: Result = {2} -3,1,7,12,15

array: {-15, -14,1,2,3,4} result = -1